Luiss
Libera Università Internazionale
degli Studi Sociali Guido Carli

Computer
Data Privacy and Security

Data Science and Management (DASMA)

LUISS

What is a Computer?

A computer is a machine that can be programmed to carry out sequences of arithmetic or logical operations automatically.

What is a Computer?

A computer is a machine that can be programmed to carry out sequences of arithmetic or logical operations automatically.

To understand how a computer works, we need to understand two main components:

1. **Hardware**: the physical components of a computer
2. **Software**: the programs that run on the hardware

What is a Computer?

A computer is a machine that can be programmed to carry out sequences of arithmetic or logical operations automatically.

To understand how a computer works, we need to understand two main components:

1. **Hardware**: the physical components of a computer
2. **Software**: the programs that run on the hardware

And to manage the complexity of running multiple programs on the hardware, we need:

3. **Operating Systems**: software that manages hardware and software resources

What is a Computer?

A computer is a machine that can be programmed to carry out sequences of arithmetic or logical operations automatically.

To understand how a computer works, we need to understand two main components:

1. **Hardware**: the physical components of a computer
2. **Software**: the programs that run on the hardware

And to manage the complexity of running multiple programs on the hardware, we need:

3. **Operating Systems**: software that manages hardware and software resources

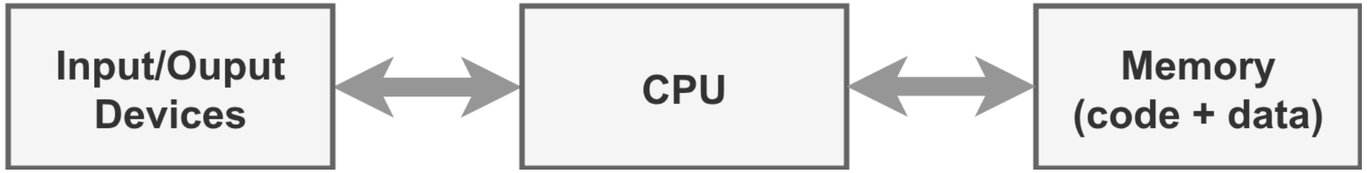
To deploy and manage these systems at scale, we may use:

4. **Virtualization**: creating multiple virtual machines on physical hardware
5. **Containerization**: packaging applications with their dependencies

Hardware

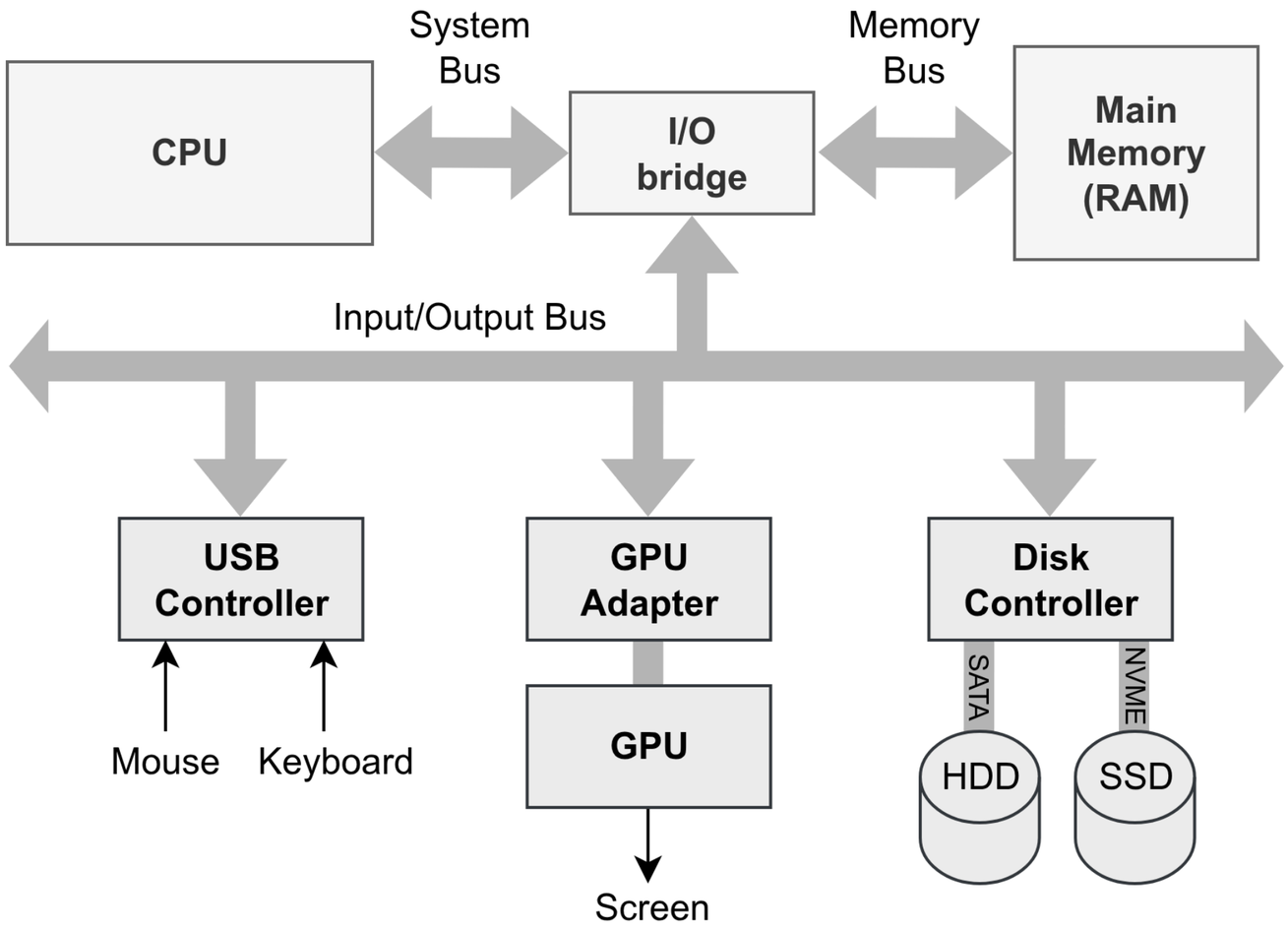
What is inside a Computer?

A (still accurate) high-level overview of a computer:

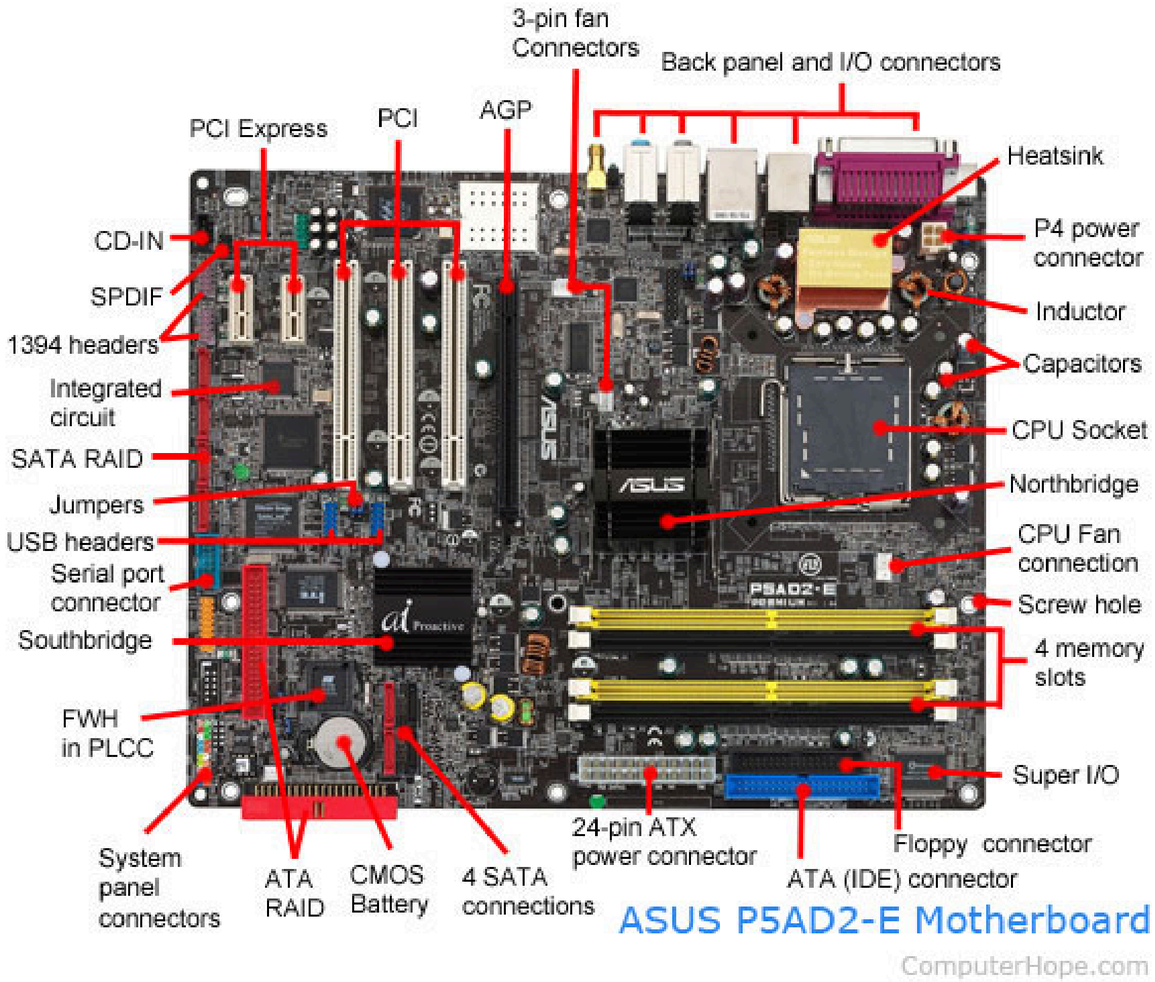


von Neumann architecture (1945)

A More Detailed Overview

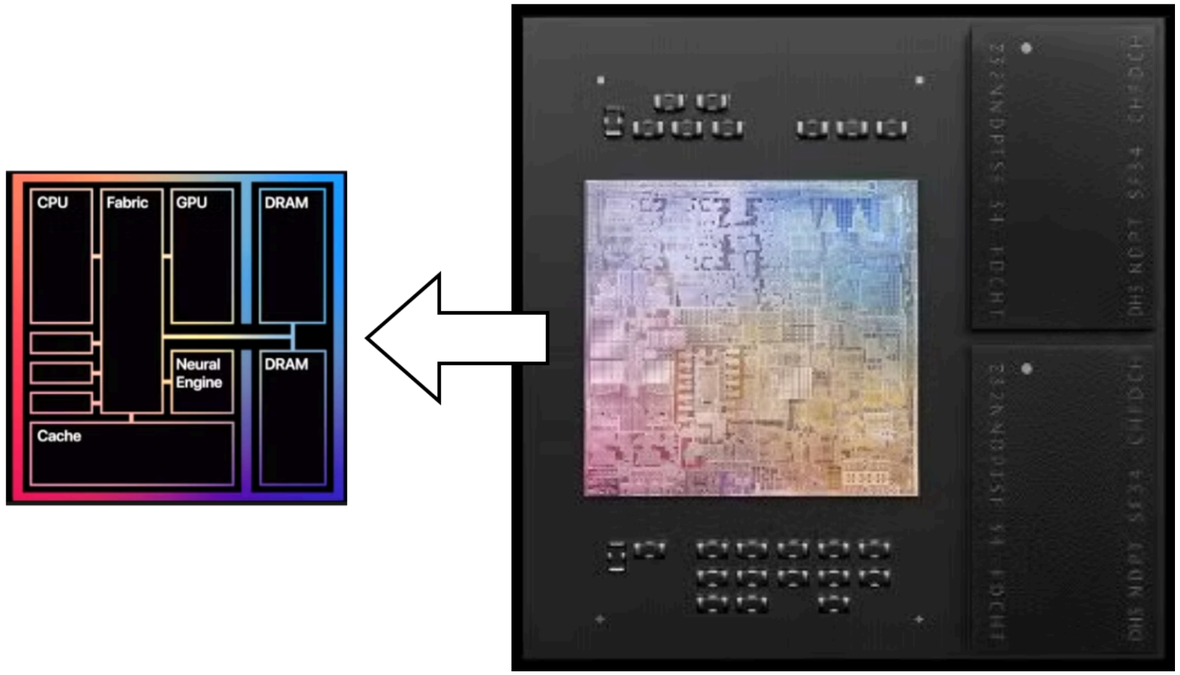


Motherboard

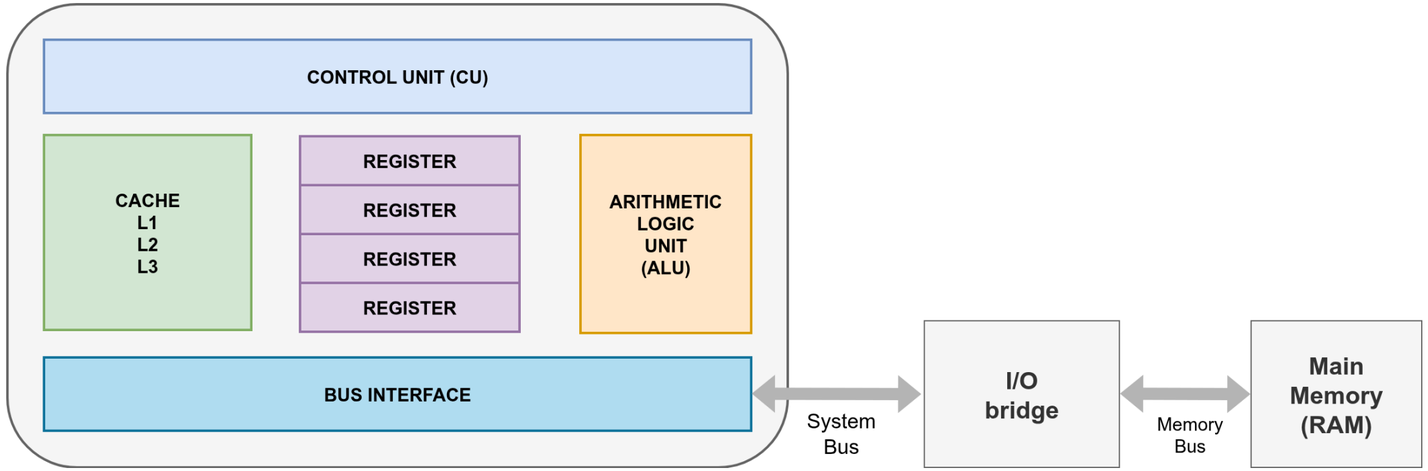


Modern System On Chip (SoC)

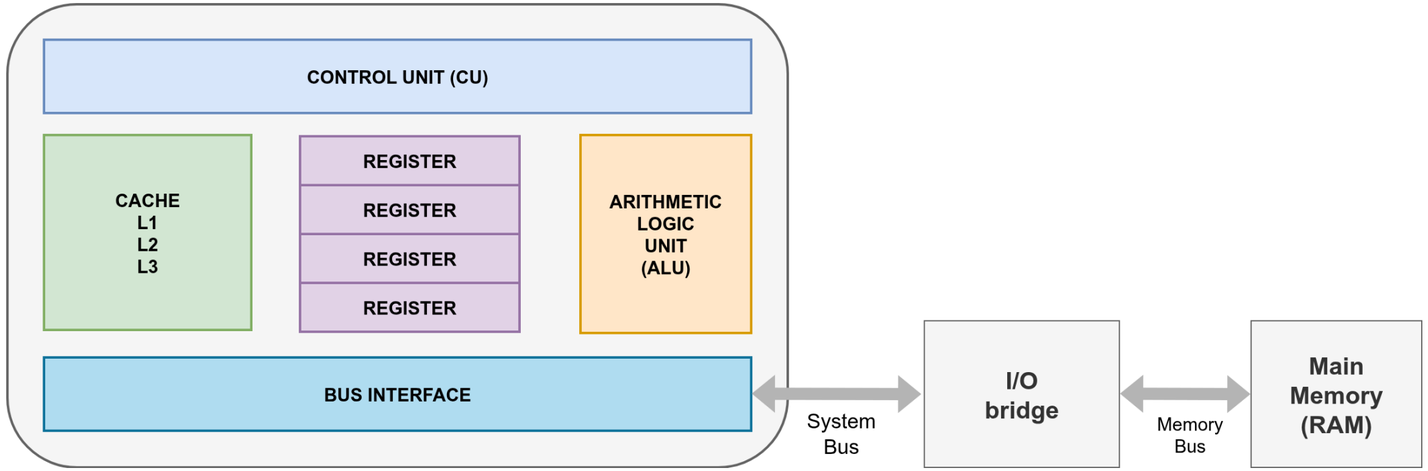
Modern SoCs integrate the CPU, GPU, storage, and other components on a single chip:



CPU (Central Processing Unit)



CPU (Central Processing Unit)



- The **Arithmetic Logic Unit (ALU)** performs arithmetic operations (such as addition and subtraction) and logical operations (like AND, OR, NOT) on binary data.
- The **Control Unit** coordinates the operations of the CPU, managing the flow of data and instructions (e.g., determining the next instruction to execute, performing the fetch-decode-execute cycle).
- The **Registers** are a few small (up to 64 bits wide), fast (<0.5 nanosecond) volatile memory locations that store data and instructions for processing.
- The **Cache** is a high-speed (1-16 nanoseconds) but limited (e.g., 1-16 MB) volatile memory that stores frequently used data to speed up access.

RAM (Random Access Memory)

RAM (Random Access Memory)

- The main (volatile) memory of the computer, where data and instructions are stored for processing before being loaded into the CPU.
- Implemented with Dynamic Random Access Memory (DRAM) chips, based on capacitors (require regular refreshes to maintain data).
- The capacity of RAM is nowadays on the order of gigabytes (GB) or terabytes (TB).
- Much faster ($1000 - 100000\times$) than secondary storage (e.g., hard drives, SSDs) but significantly slower ($10 - 100\times$) than CPU registers.
- Like a huge array of consecutive bytes, each identified by its address (e.g., $0, 1, 2, \ldots, 2^{64} - 1$).

Disks

Permanent storage devices used to store data and programs for long-term use:

Disks

Permanent storage devices used to store data and programs for long-term use:

- **Hard Drives (HDDs)**: use spinning disks to store data, quite slow speed (≈200 MB/s) but large capacity (up to 20 TB).
- **Solid-state disks (SSDs)**: use flash memory to store data, much faster speed (≈1-4 GB/s) but smaller capacity (up to 2 TB).

Disks

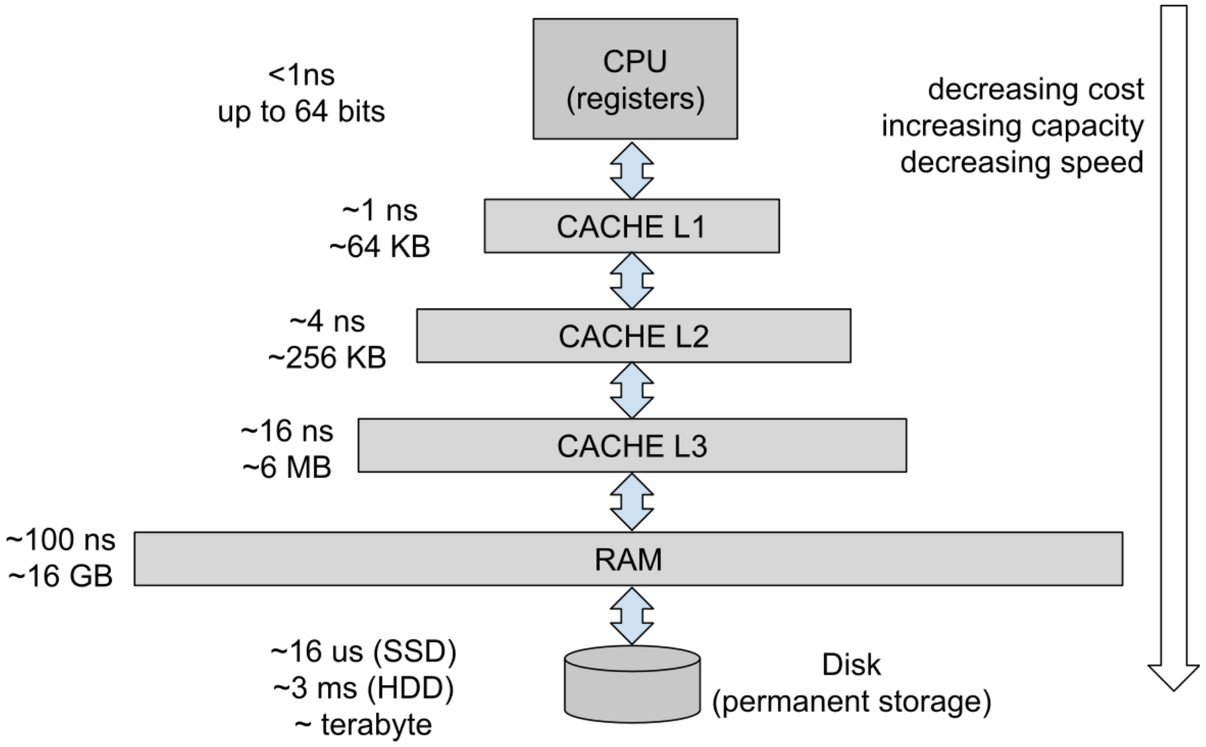
Permanent storage devices used to store data and programs for long-term use:

- **Hard Drives (HDDs)**: use spinning disks to store data, quite slow speed (⊠200 MB/s) but large capacity (up to 20 TB).
- **Solid-state disks (SSDs)**: use flash memory to store data, much faster speed (⊠1-4 GB/s) but smaller capacity (up to 2 TB).

Disks can be connected to the computer in different ways:

- USB (Universal Serial Bus): used for external disks.
- SATA (Serial Advanced Technology Attachment): used for mechanical disks and older SSDs.
- NVMe (Non-Volatile Memory Express): used for modern SSDs.

Memory Hierarchy



Fast memory is expensive, hence a complex hierarchy is used to compromise between cost and performance: larger data is stored in slower, cheaper memory, that, when needed, is loaded into faster, more expensive memory in smaller amounts.

Software

How can a Computer Execute a Program?

How can a Computer Execute a Program?

The **CPU** is responsible for executing the program code:

- It interprets the instructions one by one from the **RAM**
- The instructions must be written in **machine code**

How can a Computer Execute a Program?

The **CPU** is responsible for executing the program code:

- It interprets the instructions one by one from the **RAM**
- The instructions must be written in **machine code**

The machine code is:

- **Low level**: relatively simple operations
- **CPU specific**: different CPUs have different machine codes
- **Binary format**: humans struggle to read it

Machine Code: Intel/AMD vs ARM

A simple sequence of instructions to compute the sum of two integer numbers would look like this in machine code:

Intel/AMD 64 bit machine code

```
f3 0f 1e fa
55
48 89 e5
89 7d fc
89 75 f8
8b 55 fc
8b 45 f8
01 d0
5d
c3
```

ARM 64-bit machine code

```
ff 43 00 d1
e0 0f 00 b9
e8 0f 40 b9
e9 0f 40 b9
00 7d 09 1b
ff 43 00 91
c0 03 5f d6
```

Machine Code: Intel/AMD vs ARM

A simple sequence of instructions to compute the sum of two integer numbers would look like this in machine code:

Intel/AMD 64 bit machine code	ARM 64-bit machine code

`f3 0f 1e fa`
`55`
`48 89 e5`
`89 7d fc`
`89 75 f8`
`8b 55 fc`
`8b 45 f8`
`01 d0`
`5d`
`c3`
	`ff 43 00 d1`
	`e0 0f 00 b9`
	`e8 0f 40 b9`
	`e9 0f 40 b9`
	`00 7d 09 1b`
	`ff 43 00 91`
	`c0 03 5f d6`

- Machine code is binary (shown here in hexadecimal for readability)
- Intel/AMD processors share the same machine code format
- Other processors like ARM use entirely different machine code formats

Writing Low Level Code is Impractical

Major problems when working with machine/assembly code are:

Writing Low Level Code is Impractical

Major problems when working with machine/assembly code are:

1. Programmers have to write a lot of code to perform simple operations
2. Code is not portable across different hardware platforms
3. Programmers have to deal with hardware details
4. Programming in machine/assembly code is hard and error-prone

Writing Low Level Code is Impractical

Major problems when working with machine/assembly code are:

1. Programmers have to write a lot of code to perform simple operations
2. Code is not portable across different hardware platforms
3. Programmers have to deal with hardware details
4. Programming in machine/assembly code is hard and error-prone

Yet, CPUs only understand and run machine code.

Writing Low Level Code is Impractical

Major problems when working with machine/assembly code are:

1. Programmers have to write a lot of code to perform simple operations
2. Code is not portable across different hardware platforms
3. Programmers have to deal with hardware details
4. Programming in machine/assembly code is hard and error-prone

Yet, CPUs only understand and run machine code.

How can programmers survive?

From Low-Level to High-Level Programming

Most programmers use high-level languages, such as Python, Javascript, or PHP, to write software.

From Low-Level to High-Level Programming

Most programmers use high-level languages, such as Python, Javascript, or PHP, to write software.

> The CPU does not understand high-level languages. Never!

From Low-Level to High-Level Programming

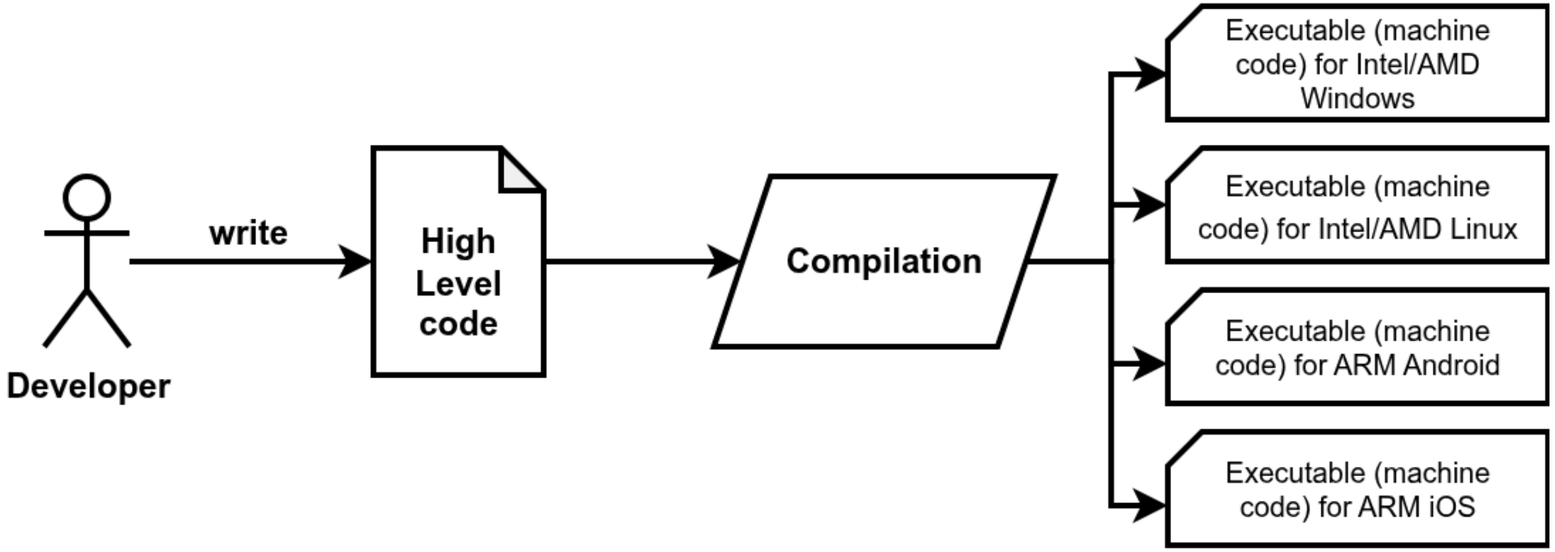
Most programmers use high-level languages, such as Python, Javascript, or PHP, to write software.

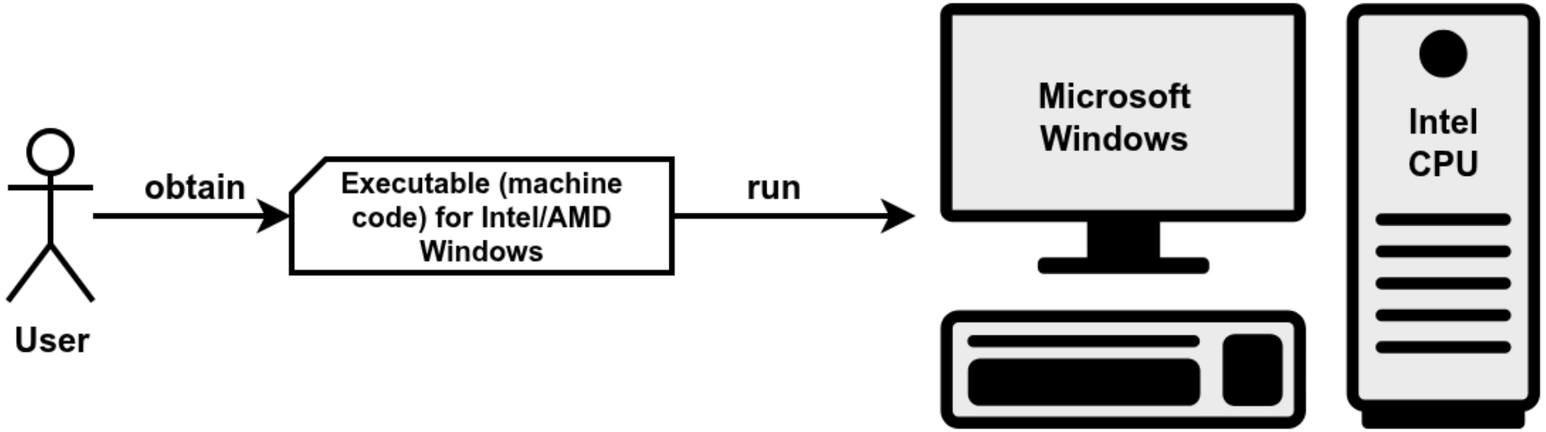
> The CPU does not understand high-level languages. Never!

Code written in high-level languages must be translated into machine code. Two main strategies:

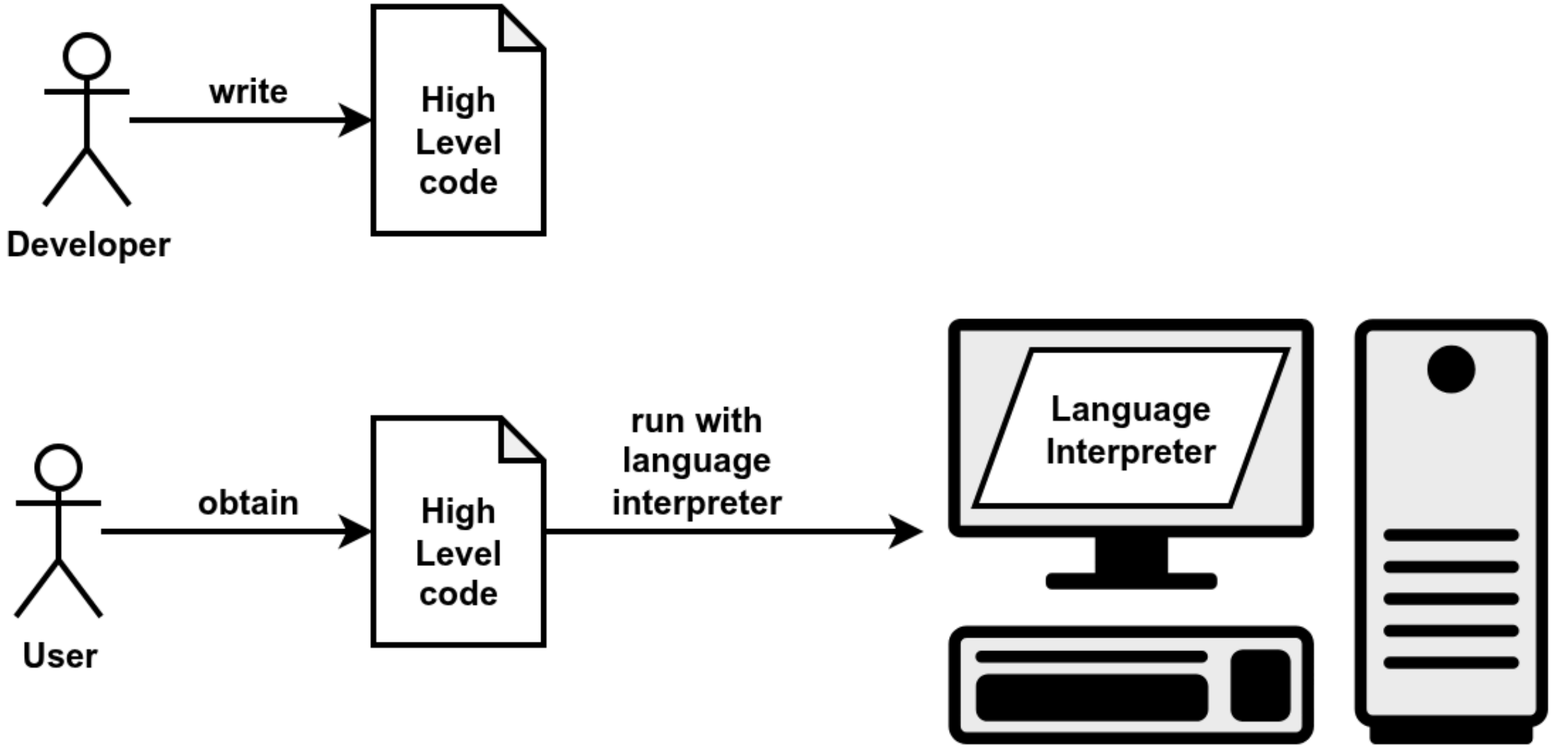
1. **Compilation**: compilers translate high-level code into machine code at *compile time* (before it is run). One executable for each platform.
2. **Interpretation**: interpreters translate high-level code into machine code at *running time*. The code is portable and platform independent.

Compilation





Interpretation



MANY Different High-Level Languages

MANY Different High-Level Languages

There are thousands of high-level languages:

- They have been designed with different goals in mind
- Some are general purpose, some are domain specific (e.g., web)
- Some gives more control to the user, some hide more details
- Some are compiled, some are interpreted

Computers Run Complex Software Stacks

Computers are expected to run **MANY** programs at the same time:

- Browser to visit web pages (e.g., Chrome)
- Office editor to write documents
- File manager to organize files
- Messaging applications (e.g., Telegram)
- Video streaming client (e.g., Netflix)
- ...and way more!

Computers Run Complex Software Stacks

Computers are expected to run **MANY** programs at the same time:

- Browser to visit web pages (e.g., Chrome)
- Office editor to write documents
- File manager to organize files
- Messaging applications (e.g., Telegram)
- Video streaming client (e.g., Netflix)
- ...and way more!

Moreover, computers are expected to:

- embrace the concept of user accounts
- provide access only after user authentication
- protect files across users and prevent hardware damage
- allow only admin users to install other programs
- handle network connections, power management

Computers Run Complex Software Stacks

Computers are expected to run **MANY** programs at the same time:

- Browser to visit web pages (e.g., Chrome)
- Office editor to write documents
- File manager to organize files
- Messaging applications (e.g., Telegram)
- Video streaming client (e.g., Netflix)
- ...and way more!

Moreover, computers are expected to:

- embrace the concept of user accounts
- provide access only after user authentication
- protect files across users and prevent hardware damage
- allow only admin users to install other programs
- handle network connections, power management

This is a **LOT** of **HARD** work: we need an **Operating System** (**OS**).

Operating Systems

Operating Systems

Modern computing systems are based on **operating systems**, which are software designed to allow the execution of applications (programs) on the available hardware.

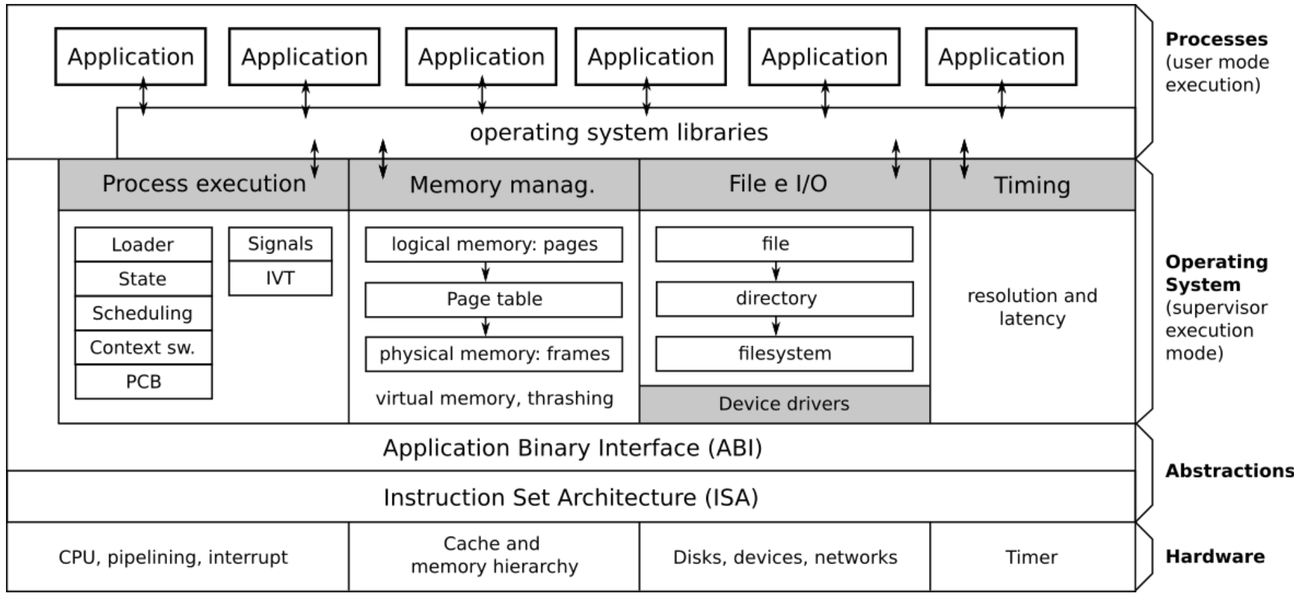
Operating Systems

Modern computing systems are based on **operating systems**, which are software designed to allow the execution of applications (programs) on the available hardware.

The main tasks of an operating system are:

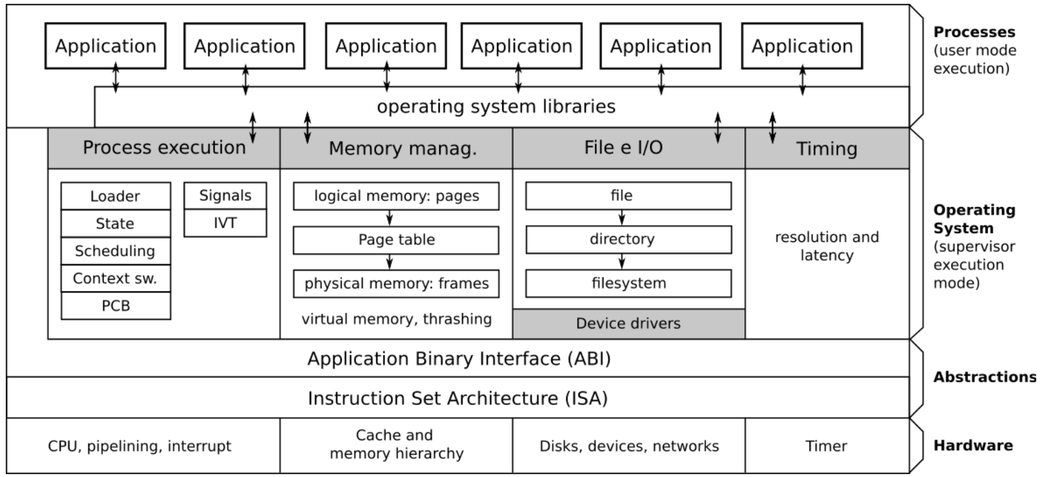
- **Process management** (creation, termination, CPU scheduling, etc.)
- **Resource allocation** to processes (RAM, storage, network connections)
- **User management** for multi-user systems (access privileges to files, programs)
- **Inter-process communication** (locally or across networked systems)
- **Error handling** during process execution or system operations
- **Protection and security**, isolating processes to prevent interference with other processes, the OS, or hardware

Overview of an Operating System



- The **hardware** is the physical part of the computer, containing CPU, RAM, disks and other devices.
- The **Instruction Set Architecture** (**ISA**) defines machine code instruction syntax and semantics.
- The **Application Binary Interface** (**ABI**) defines conventions on how different programs can interact, reuse code, and interface with the operating system.

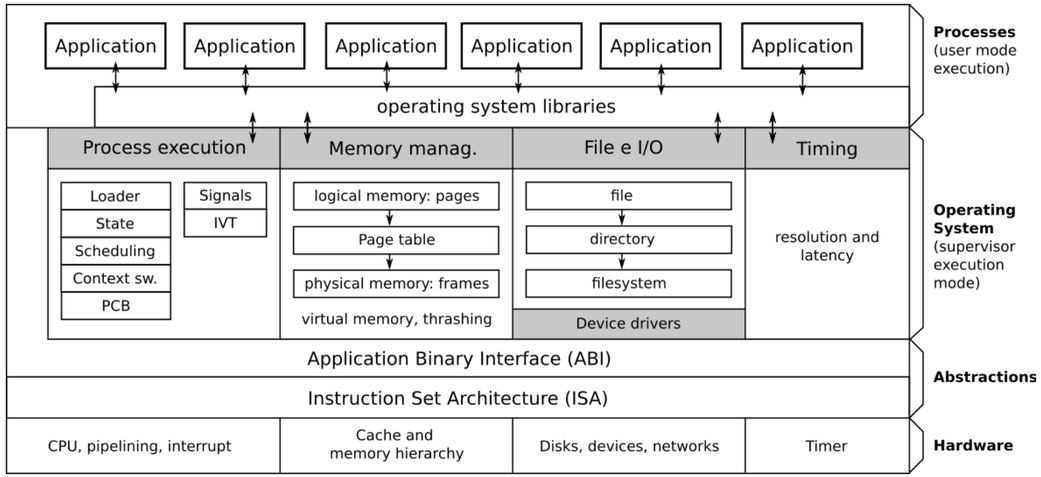
Kernel Mode vs User Mode



The **operating system**:

- has full control over the hardware: it runs in **supervisor (kernel) execution mode**
- can execute **ALL** instructions from the ISA
- controls which applications should run on the CPU cores at any time
- plays the role of intermediary between applications and the hardware

Kernel Mode vs User Mode



The **operating system**:

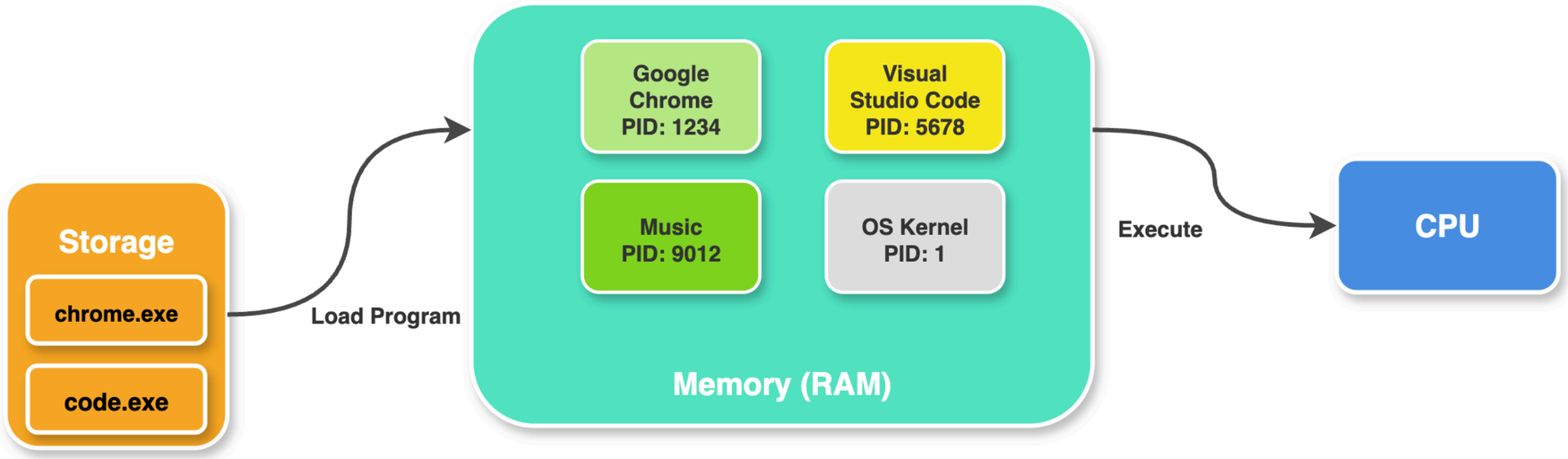
- has full control over the hardware: it runs in **supervisor (kernel) execution mode**
- can execute **ALL** instructions from the ISA
- controls which applications should run on the CPU cores at any time
- plays the role of intermediary between applications and the hardware

In **user execution mode**:

- Code can run only a limited (safe) subset of the ISA
- Privileged instructions must be delegated to the OS following the ABI
- Applications use system libraries to avoid reinventing the wheel

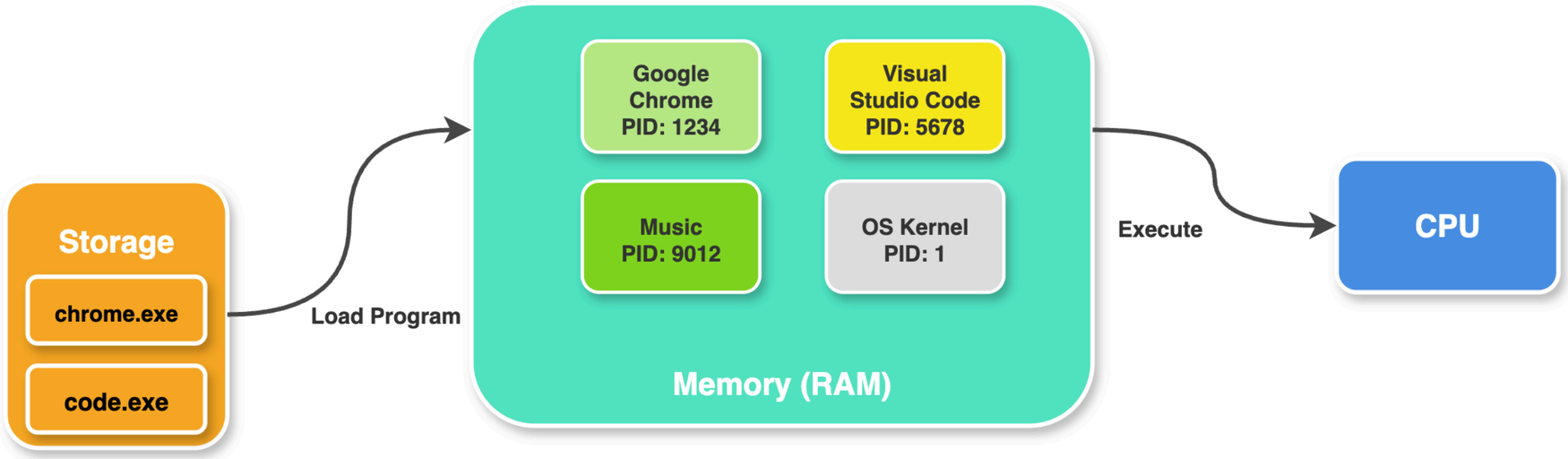
Processes

A **process** is a running program in memory:



Processes

A **process** is a running program in memory:



Key Process Concepts:

- **Process ID (PID)**: Unique identifier for each process
- **Parent/Child**: Processes can create other processes
- **Memory**: Each process has its own memory space
- **CPU Time**: Processes share CPU time

CPU Cores and Multitasking

- CPUs traditionally had only a single core (and many still have few cores)
- Yet we can run many applications simultaneously

CPU Cores and Multitasking

- CPUs traditionally had only a single core (and many still have few cores)
- Yet we can run many applications simultaneously

How is this possible?

CPU Cores and Multitasking

- CPUs traditionally had only a single core (and many still have few cores)
- Yet we can run many applications simultaneously

How is this possible?

The Operating System creates the **illusion of parallel execution**:

- CPU time is divided into small time slots (milliseconds)
- Each application gets a turn to use the CPU
- When a time slot expires, the OS performs *context switching*
- Applications take turns using the CPU so quickly that users perceive them running simultaneously

CPU Cores and Multitasking

- CPUs traditionally had only a single core (and many still have few cores)
- Yet we can run many applications simultaneously

How is this possible?

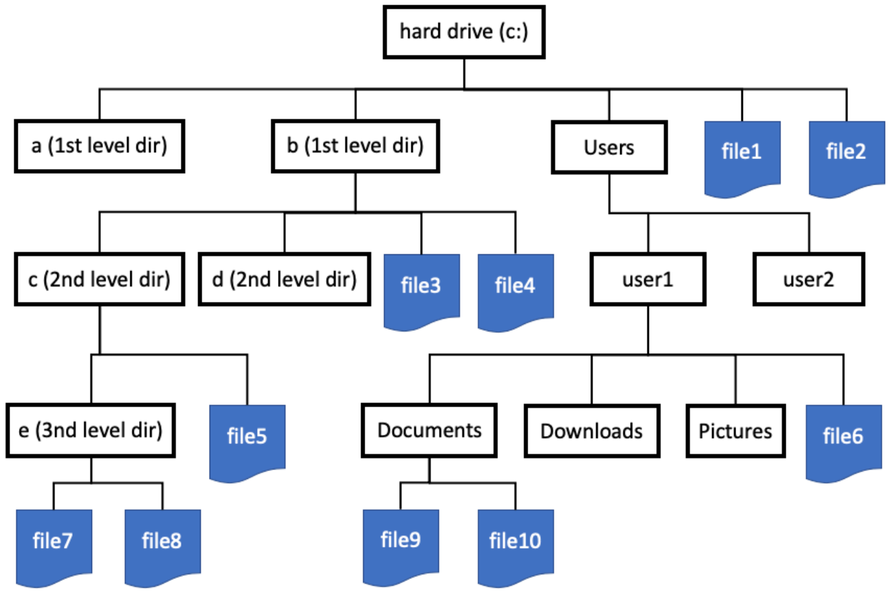
The Operating System creates the **illusion of parallel execution**:

- CPU time is divided into small time slots (milliseconds)
- Each application gets a turn to use the CPU
- When a time slot expires, the OS performs *context switching*
- Applications take turns using the CPU so quickly that users perceive them running simultaneously

This technique is called **time-sharing** or **preemptive multitasking** and it is the essential trait of all modern operating systems.

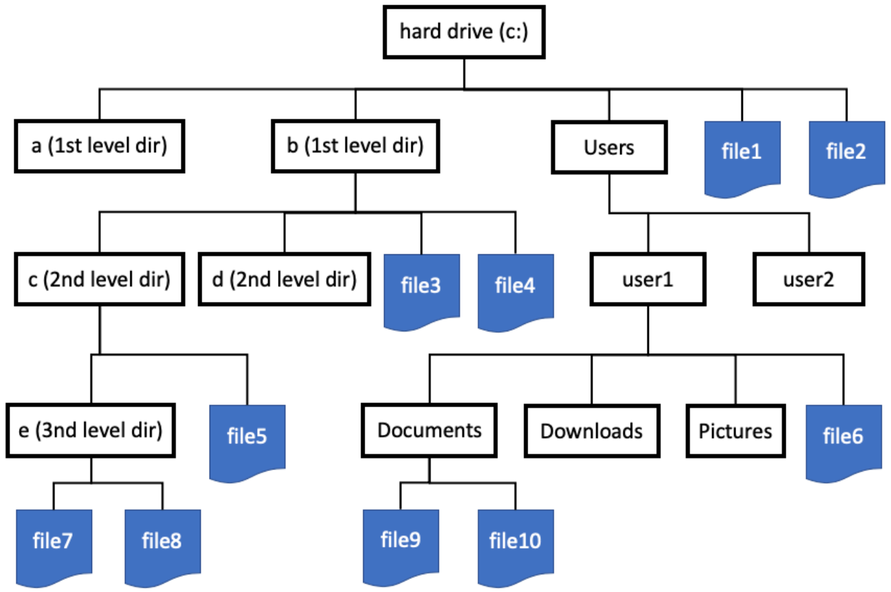
File Systems

Every operating system organizes files in a **hierarchical structure**, like a tree:



File Systems

Every operating system organizes files in a **hierarchical structure**, like a tree:



Key Concepts:

- **Root**: The top-level directory (starting point)
- **Directories/Folders**: Containers that hold files and other directories
- **Files**: Individual documents, programs, or data
- **Path**: The route to reach a specific file or directory

Application Installation Methods

Modern software can be installed and deployed in several ways:

Application Installation Methods

Modern software can be installed and deployed in several ways:

1. **Direct Downloads**: Executable files (.exe, .msi, .dmg)
2. **Application Stores**: Curated marketplaces
3. **Containers & Virtual Machines**: Isolated environments

Application Installation Methods

Modern software can be installed and deployed in several ways:

1. **Direct Downloads**: Executable files (.exe, .msi, .dmg)
2. **Application Stores**: Curated marketplaces
3. **Containers & Virtual Machines**: Isolated environments

Each method has different **goals**, **advantages**, and **trade-offs** depending on:

- Security requirements
- Deployment complexity
- Resource constraints
- Maintenance needs

Virtual Machines

Why Virtual Machines?

Sometimes we need to run software that is not compatible with our operating system, or we need to isolate applications from each other.

Why Virtual Machines?

Sometimes we need to run software that is not compatible with our operating system, or we need to isolate applications from each other.

Key motivations:

- **Isolation**: Applications run in their own environment without interfering with the host system
- **Compatibility**: Run software designed for a different OS
- **Security**: Containment of potential security vulnerabilities within the VM boundary
- **Reproducibility**: Ensures consistent behavior regardless of where the application runs

Why Virtual Machines?

Sometimes we need to run software that is not compatible with our operating system, or we need to isolate applications from each other.

Key motivations:

- **Isolation**: Applications run in their own environment without interfering with the host system
- **Compatibility**: Run software designed for a different OS
- **Security**: Containment of potential security vulnerabilities within the VM boundary
- **Reproducibility**: Ensures consistent behavior regardless of where the application runs

A **Virtual Machine (VM)** is a software emulation of a physical computer that runs a complete operating system.

How Virtual Machines Work

A **hypervisor** (or Virtual Machine Monitor) is the software layer that creates and manages VMs:

How Virtual Machines Work

A **hypervisor** (or Virtual Machine Monitor) is the software layer that creates and manages VMs:

- **Type 1 (Bare-metal)**: Runs directly on hardware (e.g., VMware ESXi, Microsoft Hyper-V, Xen)
  - Used in data centers and cloud environments
  - Better performance and security
- **Type 2 (Hosted)**: Runs on top of a host operating system (e.g., VirtualBox, VMware Workstation, Parallels)
  - Used for development and testing
  - Easier to set up

How Virtual Machines Work

A **hypervisor** (or Virtual Machine Monitor) is the software layer that creates and manages VMs:

- **Type 1 (Bare-metal)**: Runs directly on hardware (e.g., VMware ESXi, Microsoft Hyper-V, Xen)
  - Used in data centers and cloud environments
  - Better performance and security
- **Type 2 (Hosted)**: Runs on top of a host operating system (e.g., VirtualBox, VMware Workstation, Parallels)
  - Used for development and testing
  - Easier to set up

Each VM includes:

- Its own **guest operating system** (can be different from the host)
- **Virtual hardware** (virtual CPU, virtual RAM, virtual disk)
- **Applications** running inside the guest OS

Virtual Machines: Pros & Cons

Advantages:

- **Complete isolation**: Full OS-level separation between VMs
- **Run any OS**: Linux on Windows, Windows on macOS, etc.
- **Snapshots**: Save and restore the entire state of a VM
- **Security testing**: Safely analyze malware or test exploits

Virtual Machines: Pros & Cons

Advantages:

- **Complete isolation**: Full OS-level separation between VMs
- **Run any OS**: Linux on Windows, Windows on macOS, etc.
- **Snapshots**: Save and restore the entire state of a VM
- **Security testing**: Safely analyze malware or test exploits

Disadvantages:

- **Heavy resource usage**: Each VM runs a full OS (GBs of RAM, disk space)
- **Slow startup**: Minutes to boot a full OS
- **Performance overhead**: Hypervisor adds a layer of indirection
- **Portability**: VM images are large and hypervisor-dependent

Containers

Why Containers?

Virtual machines are powerful but heavy. What if we need isolation without the overhead of a full OS?

Why Containers?

Virtual machines are powerful but heavy. What if we need isolation without the overhead of a full OS?

A **container** is a lightweight, standalone, executable package that includes everything needed to run a piece of software:

- Application code
- Runtime and libraries
- System tools and settings

Why Containers?

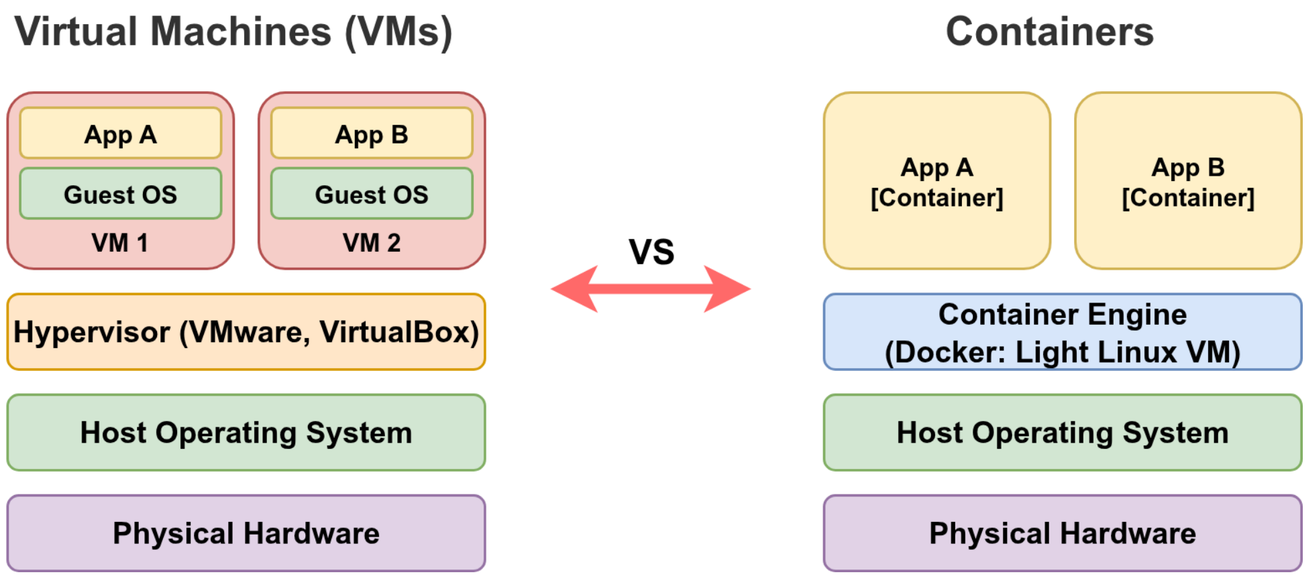
Virtual machines are powerful but heavy. What if we need isolation without the overhead of a full OS?

A **container** is a lightweight, standalone, executable package that includes everything needed to run a piece of software:

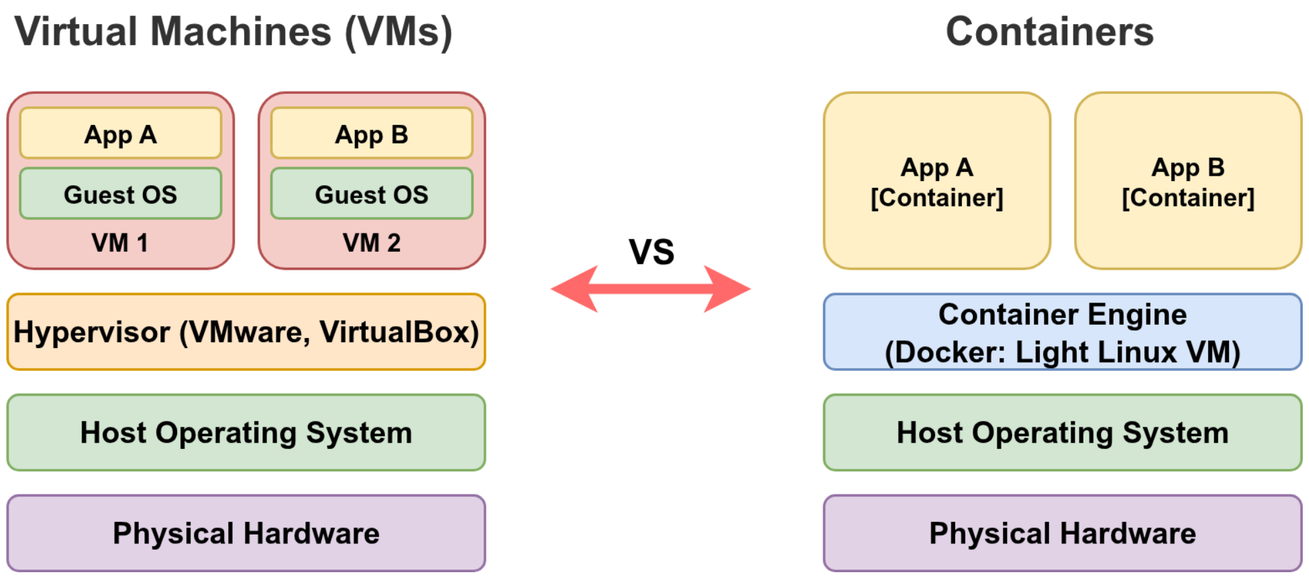
- Application code
- Runtime and libraries
- System tools and settings

Unlike VMs, containers **share the host OS kernel** and isolate the application processes from each other using OS-level features (namespaces, cgroups).

Containers vs Virtual Machines



Containers vs Virtual Machines



Aspect	Virtual Machines	Containers
Isolation	Complete OS isolation	Process-level isolation
Resource Usage	Heavy (full OS)	Lightweight (shared kernel)
Startup Time	Minutes	Seconds
Portability	Hypervisor dependent	Open format (OCI)

Containers: Pros & Cons

Advantages:

- **Lightweight**: Share the host OS kernel, minimal overhead
- **Fast startup**: Seconds instead of minutes
- **Portability**: Run consistently across different environments
- **Scalability**: Easy to deploy multiple instances
- **Dependency management**: Self-contained packages avoid conflicts

Containers: Pros & Cons

Advantages:

- **Lightweight**: Share the host OS kernel, minimal overhead
- **Fast startup**: Seconds instead of minutes
- **Portability**: Run consistently across different environments
- **Scalability**: Easy to deploy multiple instances
- **Dependency management**: Self-contained packages avoid conflicts

Disadvantages:

- **Weaker isolation**: Shared kernel means less separation than VMs
- **Linux-centric**: On macOS and Windows, Docker requires a lightweight Linux VM
- **Not ideal for GUIs**: Primarily used for non-graphical applications
- **Security concerns**: A kernel vulnerability could affect all containers

Docker: The De Facto Standard

For most users, containers means using **Docker**:

Docker: The De Facto Standard

For most users, containers means using **Docker**:

- **Docker Image**: A read-only template with instructions for creating a container (like a recipe)
- **Docker Container**: A running instance of an image (like the dish)
- **Dockerfile**: A text file with instructions to build an image
- **Docker Hub**: A public registry for sharing images

Docker: The De Facto Standard

For most users, containers means using **Docker**:

- **Docker Image**: A read-only template with instructions for creating a container (like a recipe)
- **Docker Container**: A running instance of an image (like the dish)
- **Dockerfile**: A text file with instructions to build an image
- **Docker Hub**: A public registry for sharing images

Containers have become the industry standard for deploying services in cloud environments, especially web services, due to their lightweight nature and efficient resource utilization.